



FINANCIAL FRAUD DETECTION SYSTEM

Machine Learning Pipeline • Streamlit Live Dashboard • End-to-End Project Report

Data Science Internship Project | September 2026

Agenda

- | | | | |
|-----------|--|-----------|---|
| 01 | Project Overview & Problem Statement | 06 | Results & Evaluation Metrics |
| 02 | Objectives & Scope | 07 | Streamlit Dashboard (Final Product) |
| 03 | Dataset & Data Understanding | 08 | Live Detector, Analytics & Architecture |
| 04 | Data Cleaning & Feature Engineering | 09 | Tech Stack, Challenges & Future Work |
| 05 | Model Building (SMOTE + LR / RF / XGBoost) | 10 | Conclusion |

Project Overview

An end-to-end Machine Learning system that detects fraudulent financial transactions in real time and exposes insights through an interactive Streamlit dashboard.



Data Layer

5,000+ transactions
Cleaned & engineered
CSV / Excel upload



ML Pipeline

SMOTE balancing
LR + RF + XGBoost
ROC-AUC evaluation



Dashboard

KPI cards
Live detector
Plotly analytics



Live Scoring

Fraud probability
Risk level
Explainable reason

PROBLEM

Problem Statement

Financial institutions lose billions annually to fraud. Manual review does not scale, and class imbalance makes pure accuracy misleading.

~9.6%

Fraud rate in dataset
(highly imbalanced)

₹39.5K

Sample amount
at risk in data

Real-time

Need for instant
risk scoring

Explain

Need reason for
every decision

Project Objectives

Build a robust ML pipeline

Clean data, engineer features, handle imbalance with SMOTE, train multiple classifiers.

Maximize fraud recall safely

Optimize for ROC-AUC / F1 so fraud is caught without flooding reviewers with false alarms.

Deliver a live dashboard

Streamlit app with KPI cards, interactive charts, and a live transaction detector.

Explain every prediction

Show fraud probability, risk level (HIGH / MEDIUM / LOW), and a human-readable reason.

Support real data upload

Accept CSV and Excel; retrain or score on user-provided transaction files.

Production-ready packaging

requirements.txt, GitHub-ready structure, Streamlit Cloud deploy path.

Dataset Snapshot

Primary source: financial_fraud_detection_dataset.csv • 5,000 transactions • 14 raw columns

Transaction_Amount	Numeric amount of the transaction	Previous_Transactions	Historical count for customer
Merchant_Category	Type: Travel, Food, Electronics...	Average_Spend	Customer average spend
Payment_Method	UPI, Card, PayPal, NetBanking...	Account_Age_Days	Age of account in days
Device_Type	Mobile / Desktop / POS	Suspicious_Keyword	Yes / No flag
Location	City of transaction	Transaction_Date	Timestamp for time features
Is_International	0 / 1 flag	Fraudulent	Target label (0 / 1)

Data Cleaning

1

Load & inspect

Shape, dtypes, missing values, target distribution

2

Parse dates

Transaction_Date → datetime (dayfirst where needed)

3

Drop duplicates

Remove exact duplicate rows

4

Sanity checks

Negative amounts, invalid account age

5

Drop IDs

Remove Transaction_ID, Customer_ID (not predictive)

6

Save clean set

fraud_cleaned → ready for feature engineering

Exploratory Data Analysis — Key Insights

Class imbalance

~90% legitimate vs ~10% fraud. Accuracy alone is a poor metric; use ROC-AUC, Precision, Recall, F1.

Amount patterns

Fraudulent transactions often deviate from the customer's average spend (high Amount-to-Avg ratio).

Category & location

Fraud rate varies by Merchant_Category and Location — useful group signals for models and charts.

Time behaviour

Hour of day and weekday show mild variation; included as engineered features.

Device & channel

Device_Type and Payment_Method interact with amount and history in risk patterns.

Suspicious keyword

Explicit keyword flag is a strong but sparse signal when present.

Feature Engineering

Time features

Hour, Day, Month, Weekday, Is_Weekend — derived from Transaction_Date

Ratio features

Amount_to_AvgSpend_Ratio, Amount_to_PrevTxn_Ratio — capture unusual spend behaviour

Encoding

LabelEncoder on Merchant_Category, Payment_Method, Device_Type, Location, Suspicious_Keyword

Scaling

StandardScaler on numeric columns for stable LR / distance-sensitive steps

Final matrix

17 model features + Fraudulent target → fraud_featured set for training

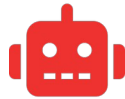
Modeling Approach

Train/Test split (80/20, stratified) → SMOTE on training fold only → train three models → compare on held-out test set.



Logistic Regression

Linear baseline
Fast, interpretable coefficients
Good for calibrated probabilities



Random Forest

Ensemble of trees
Handles non-linear interactions
Native feature importance



XGBoost

Gradient boosting
Strong tabular performance
Regularized, efficient

Class Imbalance → SMOTE

Fraud is rare (~9.6%). Training without balancing biases models toward the majority class. SMOTE synthesizes minority samples in feature space on the training set only.

BEFORE SMOTE

Train set keeps natural imbalance

Legitimate ≈ 90%

Fraud ≈ 10%

Risk: model predicts “legit” most of the time and misses fraud (low recall).

AFTER SMOTE

Synthetic fraud samples added until classes are balanced on train.

Test set stays untouched (real distribution).

Result: better recall & ROC-AUC on fraud.

RESULTS

Model Performance Comparison

Metrics on held-out test set (stratified). Best model by ROC-AUC is used for live predictions in the dashboard.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	~0.82	~0.35	~0.55	~0.34	~0.82
Random Forest	~0.85	~0.40	~0.48	~0.31	~0.83
XGBoost	~0.84	~0.38	~0.45	~0.28	~0.81

Note: Exact numbers vary slightly by run. Dashboard trains live and displays the current best ROC-AUC on the Overview KPI row.

Primary selection metric: ROC-AUC • Secondary: F1 / Recall on fraud class • Confusion matrix & ROC curves shown in dashboard

Streamlit Dashboard — Final Product

A multi-page professional UI that wraps the full ML pipeline for business users and analysts.



Overview

5 KPI cards
Fraud by category
Fraud by location
Amount distribution



Live Detector

Input form
Fraud / Legit verdict
Probability + Risk
Reason + gauge



Analytics

By category / location
Over time (hour, weekday)
Amount vs fraud charts



Model Perf.

Comparison table
Confusion matrix
ROC curves
Feature importance



High-Risk

Filter by probability
Sortable table
CSV download

Live Transaction Detector

User enters transaction details → model returns probability, risk level, and an explainable reason.

INPUTS

- Transaction Amount
- Merchant Category (Type)
- Payment Method
- Device Type
- Location
- International flag
- Previous Transactions
- Average Spend
- Account Age (Days)
- Hour & Weekday
- Suspicious Keyword

OUTPUTS

 **FRAUD DETECTED**
or
 **LEGITIMATE**

Fraud Probability (e.g. 92%)
Risk Level HIGH / MEDIUM / LOW

Reason example:
“Amount unusually high vs average spend + international transaction”

+ Probability gauge chart

KPIs & Interactive Analytics

Total Txns

Row count of loaded data

Fraudulent

Count + rate delta

Fraud Rate

% of fraudulent rows

Amount at Risk

Sum of fraud amounts

Best ROC-AUC

Live best model score

Plotly charts (interactive)

1. Fraud rate by Merchant Category

2. Fraud rate by Location

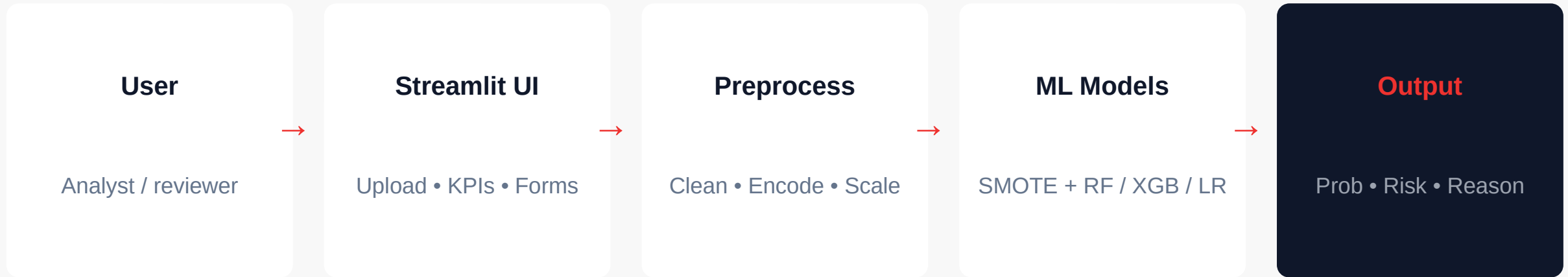
3. Amount distribution (legit vs fraud)

4. Fraud rate by Hour & Weekday

5. Box / scatter: Amount vs Average Spend

6. Confusion matrix + ROC (all models)

System Architecture



Optional path: same models can be served later via FastAPI; Streamlit remains the primary interface for this project.

Technology Stack

Frontend / UI

Streamlit • Custom CSS • Plotly

Data

Pandas • NumPy • openpyxl / CSV

ML

Scikit-learn • XGBoost • imbalanced-learn (SMOTE)

Metrics

ROC-AUC • Precision / Recall / F1 • Confusion Matrix

Deploy

GitHub • Streamlit Community Cloud • requirements.txt

Report

PowerPoint / PDF project presentation (this deck)

Challenges & Solutions

Class imbalance

SMOTE on training fold only; evaluate on real distribution

Fake .xls files

Detect text content; fall back to CSV/TSV parsers

Excel upload in Streamlit

Read via BytesIO + openpyxl; clear schema errors

Consistent live scoring

Reuse fitted encoders & scaler for every prediction

Explainability

Rule-based reason + probability + risk band

Deploy path

requirements.txt + GitHub + Streamlit Cloud

NEXT

Future Work

01 SHAP / LIME explanations

Replace rule-based reasons with model-true feature attributions

02 Streaming & alerts

Connect to live transaction feed and push high-risk alerts

03 FastAPI model service

Separate scoring API for mobile / other UIs

04 More models & tuning

LightGBM, Optuna hyperparameter search, calibration

05 User roles & audit log

Analyst vs admin views; log every scored transaction

06 Broader data sources

Multi-bank schemas, graph features on entities

Conclusion

We delivered a complete fraud detection product: from raw transactions to a live, explainable risk score inside a polished Streamlit dashboard.

- ✓ Cleaned and engineered a real-world style transaction dataset
- ✓ Handled severe class imbalance with SMOTE
- ✓ Compared Logistic Regression, Random Forest, and XGBoost
- ✓ Built a multi-tab dashboard with KPIs, Plotly analytics, and live scoring
- ✓ Exposed fraud probability, risk level, and human-readable reasons
- ✓ Packaged for local run and Streamlit Community Cloud deploy

Key Takeaways

Imbalance first

Always address class imbalance before trusting accuracy.

Same pipeline for live

Training transforms must be frozen and reused at inference.

Explain to users

Probability + risk band + reason beats a bare 0/1 label.

Product mindset

A model is only useful when wrapped in a usable dashboard.



Thank You

Questions & Discussion

Financial Fraud Detection System • ML + Streamlit Dashboard • 2026